

Análisis de Sistemas. OCL.

A continuación, se incluye el material del Capítulo 25 de Arlow y Neustadt (2006). El lenguaje de restricción de objetos (OCL) es el lenguaje que le permite añadir información adicional a un modelo UML. OCL permite:

- Especificar restricciones en elementos del modelo; puede definir reglas de negocio como restricciones en elementos del modelo.
- Definir operaciones de consulta.
- Definir las precondiciones para la realización de una operación.
- Definir las postcondiciones de la realización de una operación.

Es importante entender que no puede especificar comportamiento con OCL; no es un lenguaje de acción para UML. Esto es por lo que las expresiones OCL no tienen efectos secundarios. Por lo tanto:

- OCL no puede cambiar el valor de un elemento de modelo, solamente puede consultar valores y establecer condiciones en valores.
- OCL no se puede utilizar para especificar reglas de negocio dinámicamente en tiempo de ejecución; solamente se puede utilizar para especificar reglas de negocio en tiempo de modelado.

Puede almacenar expresiones OCL en archivos asociados con su modelo UML. Cómo se realice esto depende de la herramienta de modelado determinada que utilice. También puede anexar expresiones OCL directamente a elementos de modelado UML como notas. Esto tiene la ventaja de que hace visibles las expresiones OCL en el modelo y la desventaja de que puede saturar el modelo si existen numerosas expresiones.

Existen varias razones por las que podría encontrar OCL de utilidad:

- OCL le permite ser más preciso en su modelado; esto hace que sus modelos estén menos abiertos a una mala interpretación.
- OCL permite que herramientas de modelado generen código basándose en expresiones OCL; por ejemplo, una herramienta podría generar código para aplicar restricciones OCL como precondiciones o postcondiciones de operación.

Existen varias razones por las que podría no encontrar OCL de utilidad:

- OCL es bastante difícil de leer, la sintaxis es irregular y tiene numerosas formas abreviadas extrañas.
- En este momento, pocos modeladores conocen OCL, por lo que podría encontrar que no existe audiencia para sus expresiones OCL.
- Podría no necesitar el nivel de precisión que ofrece OCL; por un modelo UML informal que se proporciona a los programadores para su elaboración, OCL podría ser excesivo.

OCL es simplemente otra herramienta en sus recursos de modelado que le puede ayudar a crear modelos UML precisos. Es de utilidad conocerlo para que pueda utilizarlo allí donde añada un valor real.

Sintaxis de expresión OCL

OCL es un lenguaje pequeño, pero tiene una sintaxis que sigue evolucionando. En particular, tiene excepciones y atajos sintácticos que pueden perderlo. La sintaxis parece ser del estilo de C++/Java con algunos elementos del estilo de Smalltalk.

La semántica del lenguaje OCL (que está formalmente definida) es independiente de cualquier sintaxis concreta. Esto puede permitir que sintaxis alternativas de OCL aparezcan con el tiempo.

A diferencia de los lenguajes de programación más importantes, OCL es un lenguaje declarativo. Esto significa que describe el resultado que desea en lugar de describir cómo alcanzar ese resultado. Lenguajes como Java y C++ son procedimentales; describe paso a paso cómo se consigue el resultado que desea. En un lenguaje de programación convencional crea programas que se ejecutan para proporcionar algún valor al usuario.

En OCL escribe expresiones que están unidas a elementos de un modelo UML para especificar o restringir el modelo de alguna forma. Este es un punto fundamental, OCL no es un lenguaje de programación, es un lenguaje de restricción. El punto importante a record es que en OCL está especificando consultas y condiciones, no comportamientos.

La forma general de una expresión OCL se ilustra en la Figura 1. En la Figura 1 utilizamos los símbolos <...> para indicar un elemento que será remplazado con el contenido apropiado, y los símbolos [...] para indicar que el elemento es opcional.

OCL es un lenguaje tipado y toda expresión OCL evalúa a un objeto de algún tipo. Las expresiones OCL se pueden desglosar en dos partes:

- El contexto de expresión.
- Una o más expresiones.

```
context [<nombreInstanciaContextual>:]<elementoDelModelo>
<tipoExpresión> [<nombreExpresión>]: <cuerpoExpresión>
...
<tipoExpresión> [<nombreExpresión>]: <cuerpoExpresión>
```

Figura 1.

Consideramos cada una de estas partes en detalle en los siguientes apartados. Utilizaremos el modelo de la Figura 2 para proporcionar un contexto para nuestras expresiones OCL de ejemplo.

@startuml

```
abstract class Cuenta {
    saldo: Real
    propietario: String
    numeroCuenta: String
    depositar(cantidad: Real)
    obtenerSaldo(): Real
    obtenerPropietario(): String
```

```

    {abstract} retirar(cantidad: Real)
  }
class CajaDeAhorro {
  retirar(cantidad: Real)
}
class CuentaCorriente {
  limiteDescubierto: Real
  retirar(cantidad: Real)
  obtenerSaldoDisponible(): Real
  obtenerDescubiertoDisponible(): Real
}
Cuenta <|-- CajaDeAhorro
Cuenta <|-- CuentaCorriente
hide circle
@enduml

```

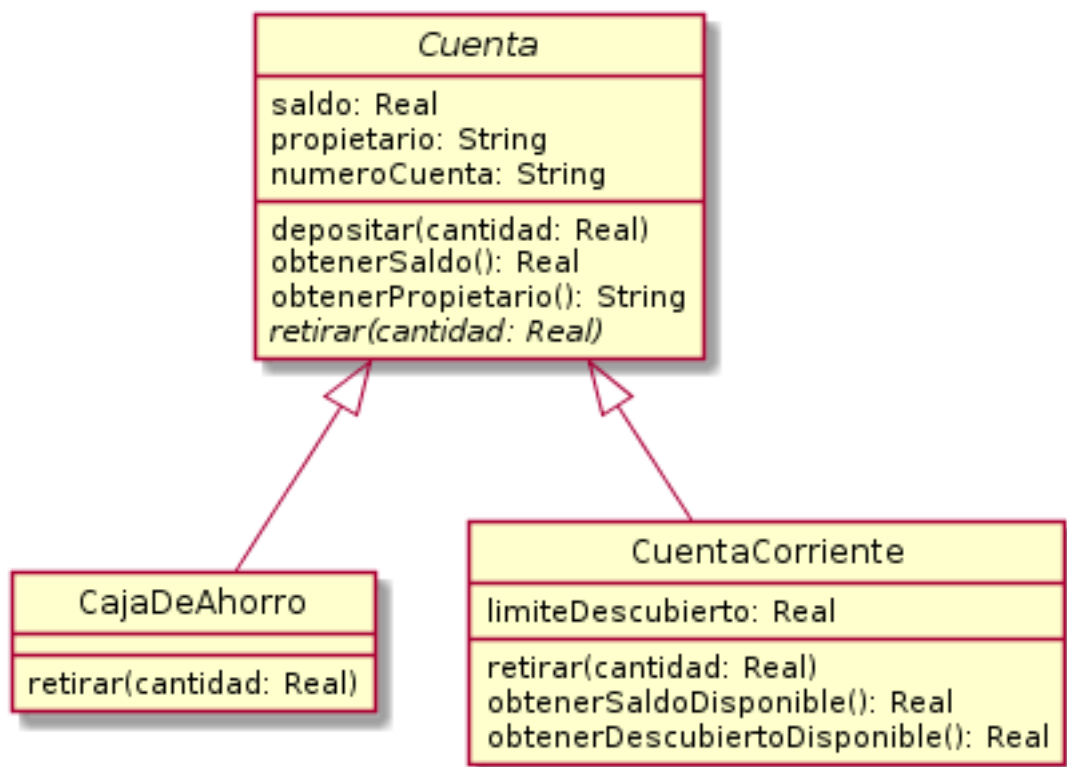


Figura 2.

El contexto de expresión

El contexto de expresión indica el elemento del modelo UML al que se une la expresión OCL. Por ejemplo, si quisiera anexar una expresión OCL a la clase **CuentaCorriente** en la Figura 2, podría definir el contexto de expresión de la siguiente forma:

context CuentaCorriente

o, podría definir el contexto como

context cuenta:CuentaCorriente

El contexto de expresión define una instancia contextual que tiene un nombre opcional (cuenta, en la primera definición no se lo definió) y un tipo obligatorio (CuentaCorriente). Piense en una instancia contextual como una instancia de la clase que puede utilizar en sus expresiones OCL. Si asigna a la instancia contextual un nombre, puede hacer referencia a ésta utilizando este nombre dentro del cuerpo de la expresión. Si no asigna a la instancia contextual un nombre, puede hacerla referencia utilizando la palabra clave OCL self. Nosotros utilizamos la palabra clave self. En la expresión anterior, la instancia contextual es una instancia de la clase CuentaCorriente a la que puede hacer referencia como cuenta o self. El tipo de instancia contextual depende del contexto de expresión.

- Si el contexto de expresión es una Clase, la instancia contextual siempre es una instancia de esa Clase.
- Si el contexto de expresión es una operación o un atributo, la instancia contextual es generalmente una instancia de la Clase que posee la operación o atributo.

Tipos de expresiones OCL

Existen ocho tipos diferentes de expresiones OCL, según se resume en la Tabla 1. Puede ver que estas expresiones se dividen en dos categorías, las que especifican restricciones (inv, pre, y post) y las que especifican atributos, operación de consulta y variables locales (init, body, def, let, y derive). En el curso sólo veremos las que especifican restricciones y las que especifican operaciones de consulta (body).

Tabla 1.

Tipo de expresión	Sintaxis	Se aplica a	Instancia contextual	Semántica
Operaciones que restringen				
invariante	inv	clase	una instancia de la clase	la invariante debe ser verdadera para todas las instancias de la clase
precondición	pre	operación	una instancia de la clase que posee la operación	la precondición debe ser verdadera antes de que se ejecute la operación
postcondición	post	operación	una instancia de la clase que posee la operación	la postcondición debe ser verdadera después de que se ejecute la operación. La palabra clave result referencia al resultado de la operación

Operaciones que definen				
operación de consulta	body	operación de consulta	una instancia de la clase que posee la operación	define la operación de consulta
valor inicial	init	atributo, extremo de asociación	atributo, extremo de asociación	define el valor inicial del atributo o extremo de la asociación
definir	def	clase	una instancia de la clase que posee la operación/variable	añade variables u operaciones de ayuda a una clase. Éstas se utilizan en expresiones OCL
variable local	let	expresión OCL	la instancia contextual de la expresión OCL	añade variables locales a expresiones OCL
valor derivado	derive	atributo, extremo de asociación	atributo, extremo de asociación	define la regla de derivación para el atributo derivado o extremo de asociación derivado

Puede asignar un <nombreExpresión> a las operaciones que restringen (inv, pre, y post). Esto le permite referenciarlas por nombre quizás para vincularlas a las especificaciones de caso de uso u a otros documentos de requisitos. Un buen estilo OCL indica que debería:

- Nombrar siempre las restricciones (aunque el nombre es opcional).
- Elegir nombres descriptivos que resuman la semántica de la restricción.
- Asegurarse de que los nombres de restricción son únicos dentro de su modelo.
- Escribir en mayúscula la primera letra de cada palabra con la primera letra de todas en minúscula para nombres de restricción.

No puede asignar un nombre de expresión a las operaciones que definen (init, body, def, let, derive).

Tratamos más adelante la semántica detallada de los diferentes tipos de expresión OCL para las operaciones que restringen y la operación de consulta.

Cuerpo de expresión

El cuerpo de expresión contiene lo más importante de la expresión OCL. Puede ver un ejemplo sencillo en la Figura 3. El contexto es CuentaCorriente, se define una invariante, y el cuerpo de expresión es `self.saldo >= self.limiteDescubierto`, es una expresión de tipo Boolean.

```
context CuentaCorriente
```

```
inv: self.saldo >= self.limiteDescubierto
```

Figura 3.

En el resto de apartados, presentamos la sintaxis OCL con idea de poder construir cuerpos de expresión OCL.

Comentarios, palabras clave y reglas de precedencia

Los comentarios se ignoran por los procesadores OCL. Utilice comentarios para documentar sus expresiones OCL para que sean más entendibles.

Una buena forma de comentar una expresión OCL es escribir la expresión en su propio idioma. OCL tiene dos estilos para comentarios:

-- Éste es un comentario de una línea. El resto de la línea detrás de los signos menos se ignora.

/ *Éste es un comentario de múltiples líneas

Cualquier cosa entre los delimitadores de comentario se ignora. * /

OCL dispone de un conjunto muy pequeño de palabras clave que no puede utilizar como nombres en expresiones OCL:

and, attr, context, def, else, endif, endpackage, if, implies, in, inv, let, not, open, or, package, post, pre, then, xor, body, init, derive .

Las operaciones OCL están sujetas a reglas de precedencia según se ilustra en la Figura 4.

mayor precedencia	::
	@pre
	. ->
	not - ^ ^^
	* /
precedencia decreciente	+ -
	if...then...else...endif
	> < <= >=
	= <>
	and or xor
menor precedencia	implies

Figura 4.

En cualquier expresión OCL, las operaciones con la precedencia mayor se ejecutan primero. Por lo tanto, por ejemplo,

$1+2*3$

evalúa como 7 porque * tiene una precedencia mayor que +.

La precedencia se puede anular con el uso de paréntesis, por lo tanto

$(1+2)*3$

evalúa como 9.

Siempre es un buen estilo en cualquier lenguaje utilizar paréntesis en lugar de confiar en las reglas de precedencia.

Tipos en OCL

OCL es un lenguaje tipado. Los tipos primitivos son Boolean, Integer, Real y String. OCL tiene también un tipo estructurado, Tuple. Además de los tipos primitivos y Tuple, OCL tiene un conjunto importante de tipos incorporados que se resumen a continuación:

- OclAny: El supertipo de todos los tipos en OCL y el modelo UML asociado.
- OclType: Una subclase de OclAny; una enumeración de todos los tipos en el modelo UML asociado.
- OclState: Una subclase de OclAny, una enumeración de todos los estados en el modelo UML asociado.
- OclVoid: El tipo “null” en OCL. Tiene una sola instancia denominada OclUndefined.
- OclMessage: Representa un mensaje.

Un aspecto crucial del sistema de tipo OCL es que todos los clasificadores en el modelo UML asociado se convierten en tipos en OCL. Esto significa que las expresiones OCL pueden hacer referencia directamente a clasificadores en el modelo asociado. Esto es lo que hace que OCL funcione como un lenguaje de restricción.

En OCL todo tipo es un subtipo de OclAny. Los tipos primitivos son subtipos directos de OclAny, mientras que los tipos del modelo UML son subclases de OclType, que a su vez es una subclase de OclAny. Todo tipo hereda el pequeño conjunto de operaciones de utilidad que se resumen en la Tabla 2.

Tabla 2.

Operación en OclAny	Semántica
Operaciones de comparación	
a = b	verdadero si a es el mismo objeto que b, de lo contrario es falso
a <> b	verdadero si a no es el mismo objeto que b, de lo contrario es falso
a.oclIsTypeOf(b:OclType): Boolean	verdadero si a es el tipo especificado por b, de lo contrario es falso
a.oclIsKindOf(b:OclType): Boolean	verdadero si a es el tipo especificado por b, o un subtipo de b, de lo contrario es falso
a.oclInState(b:OclState): Boolean	verdadero si a está en el estado b, de lo contrario es falso
a.oclIsUndefined(): Boolean	verdadero si a = OclUndefined
Operaciones de consulta	
A::allInstances():Set(A)	es una operación de ámbito de clase que devuelve un Set de todas las instancias de la

	clase A
a.oclIsNew():Boolean	verdadero si a se ha creado por la ejecución de la operación. Sólo se puede utilizar en expresiones de postcondiciones
Operaciones de conversión	
a.oclAsType(SubType):SubType	evalúa en a como un nuevo tipo en SubType. a sólo se puede asignar a uno de sus subtipos o supertipos. La asignación a un supertipo permite el acceso a propiedades redefinidas del supertipo.

allInstances() es una operación del ámbito de clase (se aplica directamente a la clase, en lugar de a cualquier instancia específica) y devuelve el Set de todas las instancias de esa clase en existencia cuando se invoca la operación.

Tipos primitivos

Los tipos primitivos de OCL son Boolean, Integer, Real y String. Estos tienen la misma semántica que en cualquier otro lenguaje (Tabla 3).

Tabla 3.

Tipo primitivo OCL	Semántica
Boolean	puede adoptar el valor true o false
Integer	un número entero
Real	un número en coma flotante
String	Una secuencia de caracteres, van entre comillas simples, por ejemplo 'Juan'

Puesto que OCL es un lenguaje de modelado en lugar de un lenguaje de programación, la especificación OCL no pone ningún límite en la longitud de los String, el tamaño de los Integer y el tamaño y precisión de Real.

Boolean

El tipo Boolean tiene dos valores, true y false. Tiene un conjunto de operaciones que devuelven valores Boolean. Las operaciones binarias están resumidas en la Tabla 4. Esta tabla muestra los resultados de las operaciones booleanas para los valores de entrada a y b.

Todas estas operaciones deberían serle familiar de otros lenguajes de programación excepto por implies. Esto procede de la lógica formal y consta de una premisa, a, y una conclusión, b. El resultado de la operación es true cuando la premisa y la conclusión tienen el mismo valor, o cuando la premisa es false y la conclusión es true. Es false cuando la premisa es true y la conclusión es false (a implies b es equivalente a: not a or b).

Tabla 4.

a	b	a=b	a<>b	a and b	a or b	a xor b	a implies b
true	true	true	false	true	true	false	true
true	false	false	true	false	true	true	false
false	true	false	true	false	true	true	true
false	false	true	false	false	false	false	true

Existe también un operador unario not que se muestra en la tabla 5.

Tabla 5.

a	not a
true	false
false	true

Las expresiones booleanas a menudo se utilizan en expresiones if...then...else...endif según la siguiente sintaxis:

```
if <expresiónBooleana> then
    <expresiónOCL>
else
    <expresiónOCL>
endif
```

Integer y Real

Integer representa un número entero y Real representa un número en coma flotante. No existe límite en la longitud de los enteros o en la longitud o precisión de los números flotantes. Integer y Real tienen el conjunto habitual de operaciones infijas aritméticas con la semántica estándar:

=, <>, <, >, <=, >=, +, -, *, /

También tienen las operaciones que se describen en la Tabla 6.

Tabla 6.

Sintaxis	Semántica	Se aplica a
a.mod(b)	el resto después de que a se divide entre b. Por ejemplo, a=8, b=3, a.mod(b) = 2	Integer
a.div(b)	el número de veces que b encada completamente dentro de a. Por ejemplo, a=8, b=3, a.div(v)=2	Integer
a.abs()	el valor absoluto de a	Integer y Real

a.max(b)	el mayor de a y b	Integer y Real
a.min(b)	el menor de a y b	Integer y Real
a.round()	el Integer más próximo a a. Si hay dos Integer igual de cerca, devuelve el mayor	Real
a.floor()	el Integer más próximo menor que o igual a a	Real

String

Las operaciones String de OCL son las definidas en la Tabla 7.

Tabla 7.

Sintaxis	Semántica
s1=s2	true si la secuencia de caracteres de s1 coincide con la secuencia s2, sino false
s1<>s2	true si la secuencia de caracteres de s1 no coincide con la secuencia s2, sino false
s1.concat(s2)	un nuevo String que es la concatenación de s1 y s2
s1.size()	el número entero de caracteres en s1
s1.toLowerCase()	un nuevo String con todos los caracteres de s1 en minúscula
s1.toUpperCase()	un nuevo String con todos los caracteres de s1 en mayúscula
s1.toInteger()	el valor Integer del string
s1.toReal()	el valor Real del string
s1.substring(inicio, fin)	un nuevo String que es subcadena de s1 desde el carácter en la posición de inicio al carácter en la posición de fin. inicio y fin deben ser Integer, inicio >= 1, fin <= s1.size(), inicio <= fin

Los String OCL son inmutables, esto significa que una vez creados no se pueden cambiar. Una operación como s1.concat(s2) siempre devuelve un nuevo String.

Tuplas

Tuplas son objetos estructurados que tienen una o más partes con nombres. Tuplas son necesarias porque algunas operaciones OCL devuelven múltiples objetos. La sintaxis de Tupla es la siguiente:

Tuple {nombreparte1: tipoParte1=valor1, nombreParte2: tipoParte2=valor2, ...}

El nombre y el valor de cada parte es obligatorio, su tipo es opcional, y el orden de las partes no está definido.

Aquí tiene una Tupla que representa información sobre el libro de Arlow:

```
Tuple {titulo : String= 'UML2 y el proceso Unificado' , editor : String= 'Addison Wesley' }
```

Las partes de Tupla se pueden inicializar por cualquier expresión OCL válida. En este sencillo ejemplo hemos utilizado literales String.

Accede a las partes de una Tupla al utilizar el operador punto. Por ejemplo, la siguiente expresión devuelve el valor 'Addison Wesley'.

```
Tuple {titulo: String= 'UML2 y el proceso Unificado' , editor : String= 'Addison Wesley' }. editor
```

OCL es un lenguaje de tipos, por lo que cada Tupla debe tener un tipo. Los TupleType son tipos anónimos. No tienen nombre y están definidos implícitamente cuando crea la Tupla. Sin embargo, es posible definir explícitamente un TupleType. Por ejemplo, el TupleType para la Tupla anterior se puede escribir en OCL como

```
TupleType {titulo: String, editor: String}
```

Normalmente, solamente necesita definir explícitamente un TupleType si sea crear una colección del tipo, por ejemplo,

```
Set (TupleType {titulo: String, editor: String} ) --crea un Set que puede albergar objetos Tupla
```

Colecciones OCL

OCL proporciona un conjunto bastante amplio de tipos de colección. Estos pueden albergar otros objetos incluidas otras colecciones.

Las colecciones OCL son inmutables. Esto significa que las operaciones de colección no cambian el estado de la colección. Por ejemplo, cuando invoca una operación para añadir o eliminar un elemento de una colección, esa operación devuelve una nueva colección, dejando la colección original sin cambios.

Los tipos de colecciones en OCL y su semántica se resume en la Tabla 8. Observe cómo cada uno de los tipos de colección OCL corresponde a un par de propiedades de extremo de asociación.

Tabla 8.

Colección OCL	Ordenada	Única (no se permiten duplicados)	Propiedades de extremo de asociación
Set	No	Si	predeterminado
OrderedSet	Si	Si	{ordered}
Bag	No	No	{nonunique}
Sequence	Si	No	{ordered, nonunique}

Las colecciones OCL son en realidad plantillas que se deben instanciar en un tipo antes de que se puedan utilizar. Por ejemplo, la expresión OCL

```
Set (Cliente)
```

instancia la plantilla Set en el tipo Cliente. Esto define un Set que alberga objetos del tipo Cliente. Puede instanciar colecciones OCL en cualquiera de los tipos disponibles.

Puede especificar constantes de colección con sólo enumerar sus elementos entre llaves:

```
OrderedSet { 'Lunes ' , 'Martes ' , 'Miércoles ' , ' Jueves ' , 'Viernes ' }
```

Esto instancia automáticamente la plantilla de colección en el tipo de elementos.

Secuencias de literales Integer tienen su propia sintaxis especial utilizando una especificación de intervalo:

```
<inicio>.. <fin>
```

Esto significa “todos los Integer entre <inicio> y <fin>” donde <inicio> y < fin> son expresiones OCL que evalúan como Integer. Por ejemplo:

```
Sequence{1..7} es equivalente a Sequence{1,2,3,4,5,6,7}
```

```
Sequence{2 . . (3+4) } es equivalente a Sequence{2,3,4,5,6,7}
```

Las colecciones pueden contener otras colecciones, por ejemplo

```
OrderedSet {OrderedSet{'Lunes','Martes '}, OrderedSet{ 'Miércoles',' Jueves', 'Viernes'}}
```

Operaciones de colección

Las colecciones tienen un amplio conjunto de operaciones. Estas se deben invocar con una sintaxis especial que utiliza el operador de flecha:

```
unaColección->operaciónColección(parámetros...)
```

Esta sintaxis especial es necesaria porque OCL puede tratar cualquier objeto como un Set que contiene solamente ese objeto. Por lo tanto, si el objeto tiene una operación denominada contar() , y Set tiene también una operación denominada contar() , OCL necesita alguna forma de distinguir entre las dos operaciones contar(); la que pertenece al objeto y la que pertenece a la colección. Realiza esto al invocar operaciones de objeto utilizando el operando del punto e invocando operaciones de colección utilizando el operador de flecha.

En los siguientes apartados resumimos la semántica de las operaciones de colección. Para facilitar la referencia, las hemos organizado en las siguientes categorías:

- Operaciones de conversión: Convierten un tipo de colección en otro.
- Operaciones de comparación: Comparan colecciones.
- Operaciones de consulta: Obtienen información sobre la colección.
- Operaciones de acceso: Acceden a elementos en la colección.
- Operaciones de selección: Devuelven una nueva colección que contiene un subconjunto o superconjunto de una colección.

Además, las colecciones OCL tienen un conjunto completo de operaciones de iteración. Estas son bastante complejas y tienen una sintaxis poco habitual, por lo que las tratamos aparte. Hemos introducido un par de convenciones para que nuestra explicación de las colecciones sea más sencilla y más compacta:

- X(T) : Una notación abreviada donde X puede ser Set, OrderedSet, Bag o Sequence.
- Colección destino: El objeto en el que se invoca la operación.

Cuando lea los siguientes apartados, recuerde que los tipos de colección son tipos plantilla. Esto significa que Set (T) es un Set instanciado en el tipo T. Por lo tanto, X(T) representa un Set, OrderedSet, Bag o Sequence instanciado en el tipo T.

Operaciones de conversión

Las operaciones de conversión (véase la Tabla 9) convierten una colección de un tipo en otro al devolver una nueva colección del tipo requerido. Por ejemplo,

Bag{'Jose' , 'María'}->asSet()

devuelve un nuevo Set que contiene los String 'José' y 'María'

Tabla 9. Operaciones de conversión

Operación de colección	Semántica
X(T)::asSet():Set(T) X(T)::asBag():Bag(T) X(T)::asOrderedSet():OrderedSet(T) X(T)::asSequence():Sequence(T)	convierte una colección de un tipo de colección en otro. cuando una colección se convierte a Set, los elementos duplicados se descartan. cuando una colección se convierte a OrderedSet o Sequence, el orden original se conserva, sino, si no había orden, se establece un orden arbitrario.
X(T)::flatten():X(T2)	tiene como resultado una nueva colección más plana instanciada en T2. Por ejemplo Set{Sequence{1,2},Sequence{3,4}} es del tipo Set(Sequence(Integer)) Set{Sequence{1,2},Sequence{3,4}}->flatten() es del tipo Set(Integer) y sería Set{1,2,3,4}

Operaciones de comparación

Las operaciones de comparación (Tabla 10) comparan la colección destino con una colección de parámetros del mismo tipo y devuelve un resultado booleano. Las operaciones tienen en cuenta las restricciones de orden de las colecciones.

Tabla 10. Operaciones de comparación

Operación de colección	Semántica
X(T)::=(y:X(T)):Boolean	Set y Bag: es true si y contiene los mismos elementos que la colección destino. OrderedSet y Sequence: es true si y contiene los mismos elementos en el mismo orden que la colección destino.
X(T)::<>(y:X(T)):Boolean	Set y Bag: es true si y no contiene los mismos elementos que la colección destino. OrderedSet y Sequence: es true si y no contiene los mismos elementos en el mismo orden que la colección destino.

Operaciones de consulta

Las operaciones de consulta (Tabla 11) permiten obtener información sobre la colección.

Tabla 11. Operaciones de consulta

Operación de colección	Semántica
X(T)::size():Integer	número de elementos en la colección destino
X(T)::sum():T	suma de todos los elementos en la colección destino. El tipo T debe soportar el operador +
X(T)::count(objeto:T):Integer	número de ocurrencias de objeto en la colección destino
X(T)::includes(objeto:T):Boolean	true si la colección destino contiene a objeto
X(T)::excludes(objeto:T):Boolean	true si la colección destino no contiene a objeto
X(T)::includesAll(c:Colección(T)):Boolean	true si la colección destino contiene todos los elementos en c
X(T)::excludesAll(c:Colección(T)):Boolean	true si la colección destino no contiene todos los elementos en c
X(T)::isEmpty():Boolean	true si la colección destino está vacía, sino false
X(T)::notEmpty():Boolean	true si la colección destino no está vacía, sino false

Operaciones de acceso

Solamente las colecciones ordenadas `OrderedSet` y `Sequence` permiten acceder a sus elementos directamente por posición (Tabla 12).

Tabla 12. Operaciones de acceso

Operación de colección	Semántica
OrderedSet(T)::first():T Sequence(T)::first():T	primer elemento de la colección
OrderedSet(T)::last():T Sequence(T)::last():T	último elemento de la colección
OrderedSet(T)::at(i:Integer):T Sequence(T)::at(i:Integer):T	elemento en la posición i
OrderedSet(T)::indexOf(objeto:T):Integer	posición de objeto en la colección

Operaciones de selección

Las operaciones de selección (Tabla 13) permiten obtener nuevas colecciones que son superconjuntos o subconjuntos de la colección destino.

Tabla 13. Operaciones de selección

Operación de colección	Semántica
$X(T)::\text{union}(y:X(T)):X(T)$	nueva colección que es el resultado de la unión de y y del Set destino; la nueva colección es siempre del mismo tipo que la colección destino. Los elementos duplicados se eliminan. Si es necesario se establece un orden
$\text{Set}(T)::\text{intersection}(y:\text{Set}(T)):\text{Set}(T)$ $\text{OrderedSet}(T)::\text{intersection}(y:\text{OrderedSet}(T)):\text{Set}(T)$	nueva colección que contiene elementos comunes a y y la colección destino
$\text{Set}(T)::\text{symmetricDifference}(y:\text{Set}(T)):\text{Set}(T)$ $\text{OrderedSet}(T)::\text{symmetricDifference}(y:\text{OrderedSet}(T)):\text{Set}(T)$	nueva colección que contiene elementos que existen en la colección destino e y , pero no en ambos
$\text{Set}(T)::-(y:\text{Set}(T)):\text{Set}(T)$ $\text{OrderedSet}(T)::-(y:\text{OrderedSet}(T)):\text{Set}(T)$	nueva colección que contiene todos los elementos de la colección destino que no están también en y
$X(T)::\text{product}(y:X(T_2)):\text{Set}(\text{Tuple}\{\text{first}:T, \text{second}:T_2\})$	producto cartesiano de la colección destino e y ; este es un Set de objetos $\text{Tuple}\{\text{first}, \text{second}\}$ donde first es un miembro de la colección destino y second es un miembro de y
$X(T)::\text{including}(\text{objeto}:T):X(T)$	
$X(T)::\text{excluding}(\text{objeto}:T):X(T)$	
$\text{Sequence}(T)::\text{subsequence}(i:\text{Integer}, j:\text{Integer}):\text{Sequence}(T)$	
$\text{OrderedSet}(T)::\text{suborderedSet}(i:\text{Integer}, j:\text{Integer}):\text{OrderedSet}(T)$	
$\text{OrderedSet}(T)::\text{append}(\text{objeto}:T):\text{OrderedSet}(T)$ $\text{Sequence}(T)::\text{append}(\text{objeto}:T):\text{Sequence}(T)$	
$\text{OrderedSet}(T)::\text{prepend}(\text{objeto}:T):\text{OrderedSet}(T)$ $\text{Sequence}(T)::\text{prepend}(\text{objeto}:T):\text{Sequence}(T)$	
$\text{OrderedSet}(T)::\text{insertAt}(i:\text{Integer}, \text{objeto}:T):\text{OrderedSet}(T)$	

Sequence(T):: insertAt(i: Integer, objeto:T):Sequence(T)

Operaciones de iteración

Las operaciones de iteración le permiten pasar en bucle sobre los elementos de una colección. Tienen la siguiente forma general:

unaColección-><operaciónIterador>(<variableIterador> | <expresiónIterador>)

Las operaciones de iterador funcionan de la siguiente manera:

- La operaciónIterador visita cada elemento de unaColección por vez.
- El elemento actual está representado por variableIterador.
- La expresiónIterador se aplica a la variableIterador para generar un resultado.
- Toda operaciónIterador gestiona el resultado en su propia forma particular.
- El tipo de la variableIterador es del mismo tipo que los elementos en unaColección.

La variableIterador es opcional, cuando se visita cada elemento de la colección todas sus características están automáticamente accesibles para la expresiónIterador y se pueden acceder directamente por su nombre. Por ejemplo, si el elemento es un objeto CuentaBancaria con un atributo denominado saldo, la expresiónIterador puede hacer referencia a saldo directamente. Sin embargo, omitir la variableIterador puede ser peligroso y se lo considera un mal estilo. Esto es porque la expresiónIterador primero busca su propio espacio de nombres para cualquier variable que necesite y si no puede encontrar la variable, busca en espacios de nombre cercanos. Si omite la variableIterador existe un riesgo de que la expresiónIterador encuentre el elemento erróneo.

En las Tablas 14 y 15 se resumen las operaciones de iteración. En las tablas las operaciones están agrupadas en operaciones que devuelven un valor booleano (Tabla 12), y en las que devuelven una selección de la colección (Tabla 13).

Tabla 14.

Operaciones de Iterador booleanas	Semántica
X(T)::exists(i expresiónIterador):Boolean	true si la expresiónIterador evalúa como true para al menos un valor de i, sino es false
X(T)::forAll(i expresiónIterador):Boolean	true si la expresiónIterador evalúa como true para todos los valores de i, sino es false
X(T)::forAll(i, j, n expresiónIterador):Boolean	true si la expresiónIterador evalúa como true para cada Tuple{i,j,n}, sino es false
X(T)::isUnique(i expresiónIterador):Boolean	true si la expresiónIterador tiene un valor único para cada i, sino es false
X(T)::one(i expresiónIterador):Boolean	true si expresiónIterador evalúa como true para exactamente un valor de i, sino es false

Tabla 15.

Operaciones de Iterador de selección	Semántica
X(T)::any(i expresiónIterador):T	elemento aleatorio de la colección destino para el que expresiónIterador es verdadera
X(T)::collect(i expresiónIterador):Bag(T)	un Bag que contiene los resultados de la evaluación de expresiónIterador una vez para cada I (element en la colección destino)
X(T)::select(i expresiónIterador):X(T)	colección que contiene aquellos elementos de la colección destino para los que expresiónIterador evalúa como true
X(T)::reject(i expresiónIterador):X(T)	colección que contiene aquellos elementos de la colección destino para los que expresiónIterador evalúa como false

Merece la pena examinar más detenidamente forAll() . Esta operación tiene dos formas: La primera tiene una sola variableIterador y la segunda tiene muchas. La segunda forma abrevada para muchas operaciones forAll(...) anidadas.

Por ejemplo, considere dos operaciones forAll(...) anidadas de la siguiente forma:

```
c->forAll(i | c->forAll(j | expresiónIterador) )
```

Puede escribir esto como

```
c->forAll(i , j | expresiónIterador)
```

El efecto de ambas formas es pasar sobre un conjunto de pares {i, j} que es el producto cartesiano de c consigo mismo. Un ejemplo aclarará esto. Suponga:

```
c=Set{x,y,z}
```

El producto cartesiano de c consigo mismo es el Set:

```
{{x,x},{x,y},{x,z},{y,x},{y,y},{y,z},{z,x},{z,y},{z,z}}
```

Luego, c->forAll (i, j | expresiónIterador) pasa por cada subconjunto en este Set: , y a i y j se les asigna uno de los elementos del subconjunto. Luego puede utilizar i y j en la expresión de iterador.

Todas estas operaciones de iteración son casos especiales de la operación iterate más general que examinamos a continuación.

Operación iterate

Puede realizar sus propias iteraciones personalizadas al utilizar la operación de OCL iterate. Ésta tiene la siguiente forma:

```
unaColección->iterate( <variableIterador> : <Tipo>;
                      <variableResultado> : <TipoResultado> = <expresiónIniciación> |
                      <expresiónIterador> )
```

Puede ver que al igual que la variableIterador y su Tipo (que son obligatorio en este caso) existe una variableResultado que puede tener un tipo diferente. La variableResultatado se

inicializa en el valor expresiónIniciación. La operación iterate luego ejecuta la expresiónIterador para cada miembro de unaColección, utilizando variableIterador y el valor actual de variableResultado. El resultado de evaluar expresiónIterador se convierte en el nuevo valor de variableResultado que se utilizará cuando expresiónIterador se ejecuta en el siguiente elemento de la colección. El valor de la operación iterate(...) es el valor final de la variableResultado.

Veamos un sencillo ejemplo:

```
Bag{1,2,3,4,5}->iterate(n: Integer; suma: Integer= 0 | suma + n)
```

Esta expresión es la suma de los números en Bag, en este caso 15. Esto es equivalente a:

```
Bag{1,2,3,4,5}->sum()
```

La operación iterate es el iterador más general y se puede utilizar para representar a las demás operaciones. Por ejemplo, para seleccionar todos los números positivos de un Set.

```
Set{-2,-3,1,2}->iterate(n:Integer; numPositivos:Set(Integer)=Set{|
```

```
    If n >= 0 then numPositivos->including(n) else numPositivos endif )
```

Esto es equivalente a:

```
Set{-2,-3,1,2}->select(n:Integer | n >= 0)
```

Otro ejemplo, para obtener todos los valores absolutos de un Set:

```
Set{-2,-3,1,2}->iterate(n:Integer; valoresAbsolutos:Bag(Integer)=Bag{|
```

```
    If n >= 0 then valoresAbsolutos->including(n) else valoresAbsolutos-including(-n) endif )
```

Esto es equivalente a:

```
Set{-2,-3,1,2}->collect(n:Integer | if n >= 0 then n else -n endif)
```

Navegación OCL

Navegación es el proceso por el que sigue vínculos de un objeto origen a uno o más objetos destino.

La navegación es posiblemente el área más compleja y difícil de OCL. Para escribir una expresión OCL, tiene que saber cómo navegar del contexto de expresión a otros elementos de modelo a los que necesita hacer referencia. Esto significa que debe utilizar OCL como un lenguaje de navegación.

Las expresiones de navegación OCL pueden hacer referencia a cualquiera de lo siguiente:

- Clasificadores.
- Atributos.
- Extremos de asociación.
- Operaciones de consulta.

En la especificación de OCL éstas se denominan propiedades.

Navegación dentro de la instancia contextual

Examinemos un sencillo ejemplo de navegación para acceder a características de la instancia contextual. La Figura 3 muestra una clase A que tiene un solo atributo, a1, y una sola operación, op1().

```

@startuml
class A {
    a1: String
    op1(): String
}
hide circle
@enduml

```

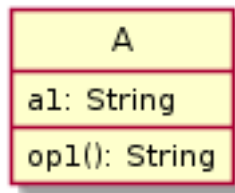


Figura 3.

Suponiendo que la clase A es el contexto de expresión, puede escribir las expresiones de navegación OCL que se listan en la Tabla 16.

Tabla 16.

Expresión de navegación	Semántica
self	la instancia contextual, una instancia de A
self.a1	el valor del atributo a1 de la instancia contextual
a1	el valor del atributo a1 de la instancia contextual
self.op1()	el resultado de op1() invocado en la instancia contextual. La operación op1() debe ser una operación de consulta
op1()	el resultado de op1() invocado en la instancia contextual. La operación op1() debe ser una operación de consulta

Existen varios puntos importantes a destacar sobre este ejemplo:

- Accede a la instancia contextual al utilizar la palabra clave self.
- Accede a las propiedades de la instancia contextual directamente o al utilizar self y el operador punto. Por cuestión de estilo, preferimos ser explícitos y utilizar self y el operador punto.
- Las únicas expresiones a las que puede acceder son operaciones de consulta.

Navegación a través de asociaciones

La navegación se hace algo más complicada cuando navega a través de asociaciones. Normalmente, puede navegar solamente través de asociaciones que son navegables, y puede

acceder solamente a características de la clase pública. La Figura 4 muestra dos clases y una asociación entre las mismas. La Tabla 17 ilustra algunas expresiones de navegación a través de la asociación entre las dos clases A y B, donde la multiplicidad en el extremo b es 1.

```
@startuml
class A {
  a1: String
}
class B {
  b1: String
  op1(): String
}
A -- "b 1" B
hide circle
@enduml
```

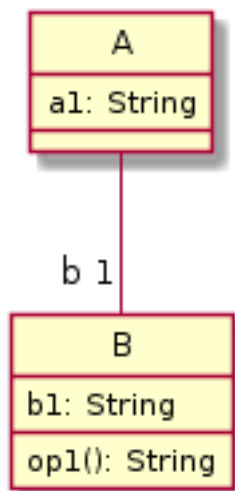


Figura 4.

Tabla 17. Expresiones de navegación (A es el contexto de expresión)

Expresión	Valor
self	instancia contextual, una instancia de A
self.b	un objeto de tipo B
b	un objeto de tipo B
self.b.b1	el valor del atributo B::b1 de un objeto de tipo B vinculado a la instancia contextual
self.b.op1()	el resultado de la operación B::op1() invocada en un objeto de tipo B vinculado a la instancia contextual

Navega a través de un extremo de asociación utilizando el operador punto como si el nombre del rol fuera un atributo de la clase de contexto. La expresión de navegación puede devolver el

objeto (u objetos) en el extremo destino, los valore de sus atributos y los resultados de sus operaciones.

La navegación se hace más complicada cuando la multiplicidad del extremo destino de la asociación es mayor que 1. Esto es porque la semántica de la navegación depende de la multiplicidad.

La Figura 5 y Tabla 18 muestran algunas expresiones de navegación a través de asociación entre dos clases, C y D, donde la multiplicidad en el extremo d es *.

```
@startuml
class C {
  c1: String
}
class D {
  d1: String
  op1(): String
}
C -- "b *" D
hide circle
@enduml
```

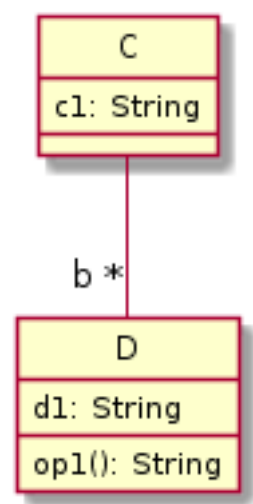


Figura 5.
Tabla 18.

Expresión	Valor
self	instancia contextual, una instancia de C
self.d	un SET(D) de objetos de tipo D
d	un SET(D) de objetos de tipo D
self.d.d1	un Bag(String) de los valores de atributo D::d1. Es una abreviatura para self.d->collect(d1)

self.d.op1()	un Bag(String) de los valores de resultados de la operación B::op1(). Es una abreviatura para self.d->collect(op1())
---------------------	---

La expresión de navegación

self.d

devuelve un Set(D) de d objetos.

Esto significa que el operador punto está sobrecargado. Cuando la multiplicidad en el extremo destino es 1 o 0..1, devuelve un objeto del mismo tipo que la clase destino. Cuando la multiplicidad es mayor que 1, devuelve un Set instanciado en la clase destino. Por defecto, el operador punto devolverá un Set de objetos cuando la multiplicidad máxima es mayor a 1. Sin embargo, puede especificar el tipo de colección que devuelve al utilizar las propiedades del extremo de asociación listadas en la Tabla 19.

Tabla 19.

Colección OCL	Propiedades de extremo de asociación
Set	predeterminado
OrderedSet	{ordered}
Bag	{nonunique}
Sequence	{ordered, nonunique}

Cuando accede a una propiedad de una colección, por ejemplo,

self.d.d1

es una abreviatura de:

self.d->collect(d1)

unaColección->collect(expresiónIterador) devuelve un Bag que contiene los resultados de evaluar expresiónIterador para cada elemento en unaColección. En el caso self.d->collect(d1), devuelve un Bag de valores del atributo d1 para cada objeto D en el Set(D) obtenido al navegar self.d.

Navegación a través de múltiples asociaciones

En principio, es posible navegar a través de cualquier número de asociaciones. Sin embargo, en la práctica, se minimiza la cantidad de navegación y usualmente se la limita a dos asociaciones como mucho. Esto es porque las expresiones de navegación largas son propensas a errores, pueden ser difíciles de entender y de mantener. Examinaremos ejemplos sencillos de navegación a través de dos asociaciones (véase la Figura 6 - Tabla 20).

@startuml

class A {

 a1: String

```
}  
class B {  
  b1: String  
}  
class C {  
  c1: String  
}  
class F {  
  f1: String  
}  
class H {  
  h1: String  
}  
class I {  
  i1: String  
}  
class L {  
  l1: String  
}  
A -- "b 1" B  
A -- "h *" H  
B -- "c 1" C  
B -- "f *" F  
H -- "i 1" I  
H -- "l *" L  
hide circle
```

@enduml

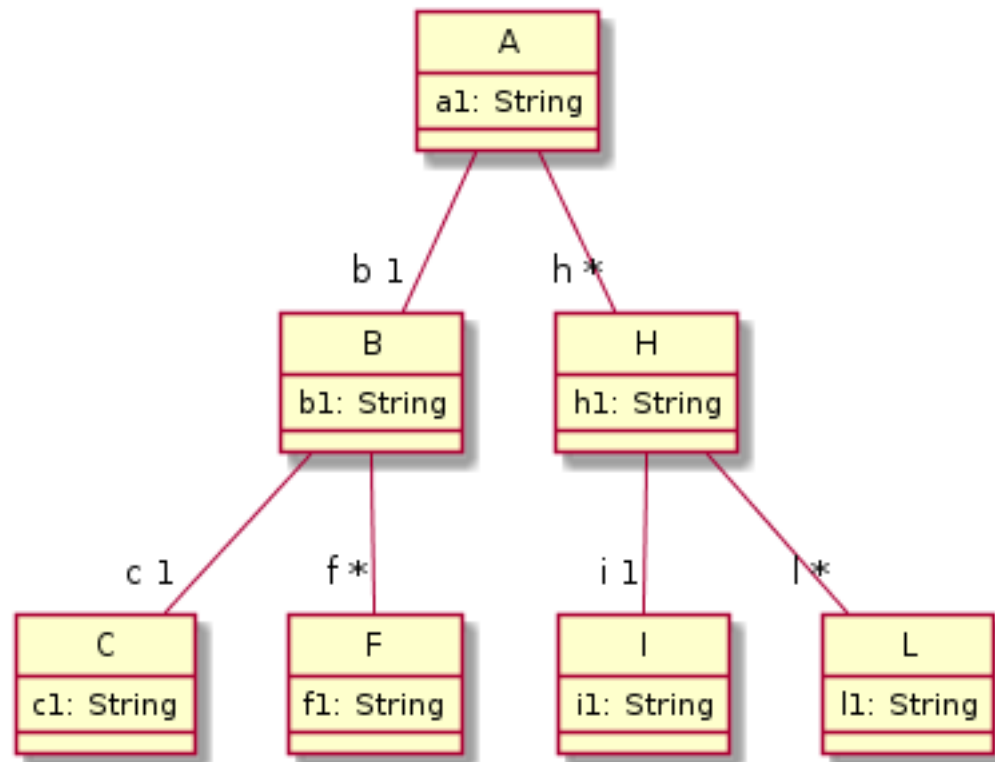


Figura 6.

Tabla 20. Expresiones de navegación (A es el context de expresión)

Expresión	Valor
self	La instancia contextual, una instancia de A
self.b	un objeto de tipo B
self.b.b1	El valor del atributo B::b1
self.b.c	Un objeto de tipo C
self.b.c.c1	El valor del atributo C::c1
self.b.f	Un Set(F) de objetos de tipo F
self.b.f.f1	Un Bag(String) de valores de atributo F::f1
self.h	Un Set(H) de objetos tipo H
self.h.h1	Un Bag(String) de valores de atributo H::h1
self.h.i	Un Bag(I) de objetos de tipo I
self.h.i.i1	Un Bag(String) de valores de atributo I::i1
self.h.l	Un Bag(L) de objetos de tipo L
self.h.l.l1	Un Bag(String) de valores de atributo L::l1

Puede ver que la navegación más allá del extremo de una asociación con multiplicidad mayor que 1 siempre tiene como resultado un Bag. Esto es porque es equivalente a aplicar collect(...). Por ejemplo, la expresión

self.h.l.l1 es equivalente a
self.h->collect(l)->collect(l1)

De forma similar, puede ampliar la navegación a través de más de dos asociaciones, pero no es habitual.

Tipos de expresión OCL en detalle

Hemos presentado los diferentes tipos de expresiones OCL anteriormente. Ahora que hemos tratado la sintaxis OCL, podemos examinar cada una de ellas en detalle. Utilizamos el modelo en la Figura 7 como ejemplo.

```
@startuml
class Persona {
    nombre: String
    id: String
    direccion: String
    obtenerNombre(): String
    obtenerId(): String
    obtenerDireccion(): String
}
abstract class Cuenta {
    saldo: Real
    numeroCuenta: String
    depositar(cantidad: Real): Real
    obtenerSaldo(): Real
    obtenerPropietario(): Persona
    obtenerOperadores(): Persona[*]
    {abstract} retirar(cantidad: Real)
}
class CajaDeAhorro {
    retirar(cantidad: Real)
}
class CuentaCorriente {
    limiteDescubierto: Real
    retirar(cantidad: Real)
    obtenerSaldoDisponible(): Real
    obtenerDescubiertoDisponible(): Real
}
Cuenta <|-- CajaDeAhorro
```

```

Cuenta <|-- CuentaCorriente
Persona "propietario 1" -- "cuentaPropietario *" Cuenta
Persona "operador 1..*" -- "cuentaOperada *" Cuenta
hide circle
@enduml

```

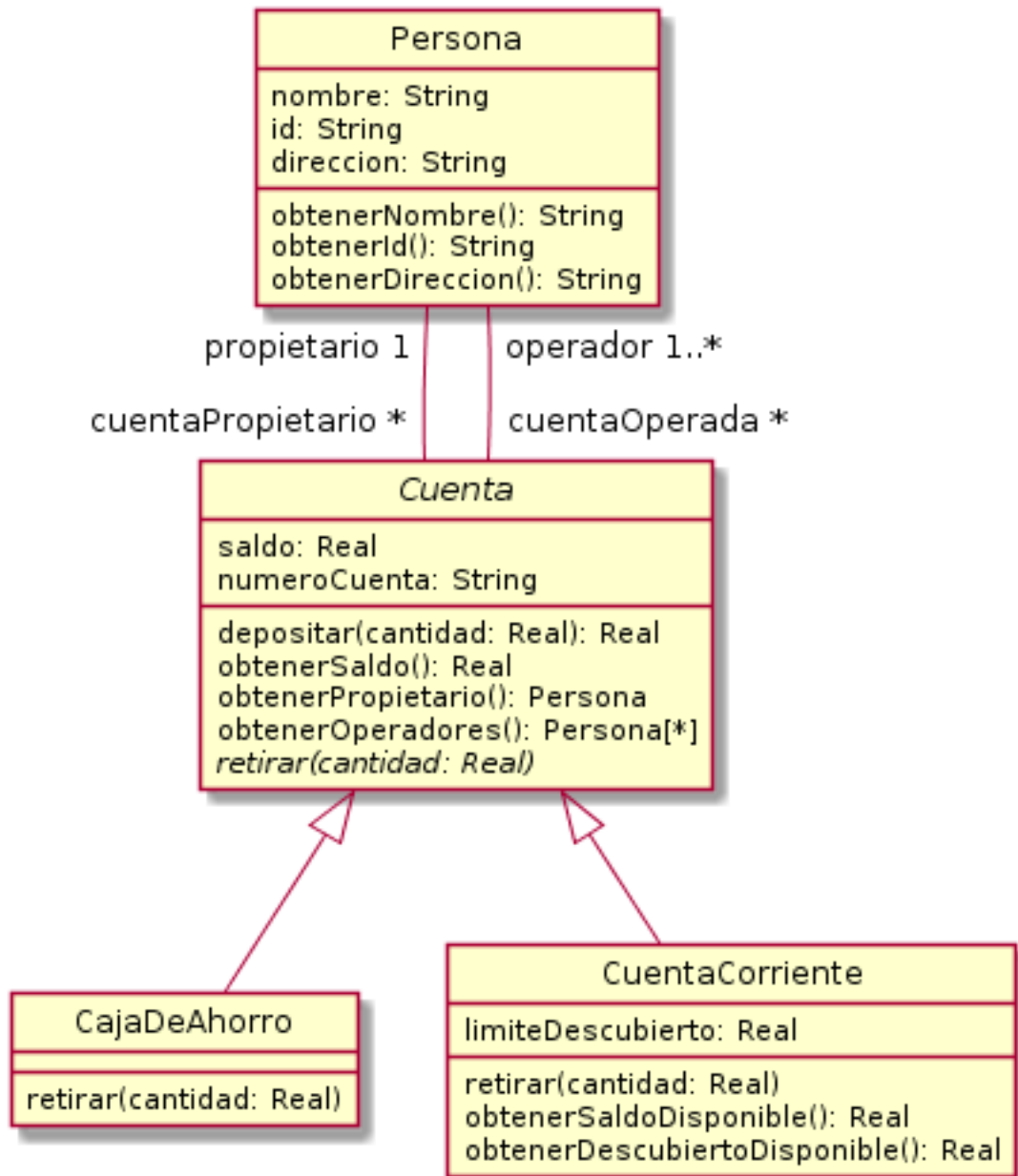


Figura 7.

Invariantes (inv)

Una invariante es una expresión que debe ser verdadera para todas las instancias de su contexto. Considere el modelo de la Figura 7, se definen las siguiente cuatro reglas de negocio sobre **CuentaCorriente** y **CajaDeAhorro**:

1. Ninguna cuenta debería estar sin fondos en más de \$ 1000.
2. CuentaCorriente tiene un crédito al descubierto. La cuenta no debería estar sin saldo en una cantidad mayor que su límite de descubierto.
3. CajaDeAhorro nunca debería estar sin saldo.
4. Todo númeroCuenta debería ser único.

La primer regla se puede expresar como una invariante de la clase Cuenta porque debe ser cierto para todas las instancias de Cuenta.

context Cuenta

inv valorSaldo:

```
-- una cuenta bancaria debería tener un saldo > -1000,0
self.saldo >= (-1000.0)
```

Esta invariante es heredada por las dos subclases, CuentaCorriente y CajaDeAhorro. Estas subclases pueden fortalecer esta invariante pero nunca debilitarla, siempre se debe mantener el principio de sustitución.

La regla 2 se puede expresar como invariante en la clase CuentaCorriente:

context CuentaCorriente

inv valorDescubierto: self.saldo >= -self.limiteDescubierto

La regla 3 se puede expresar como invariante en la clase CajaDeAhorro:

context CajaDeAhorro

inv cajaAhorroConSaldo: self.saldo >= 0.0

La regla 4 indica que todo número de cuenta debe ser único:

context Cuenta

inv numeroCuentaUnico:

```
Cuenta.allInstances()->forAll(c1, c2 | c1 <> c2 implies c1.numeroCuenta <> c2.numeroCuenta)
```

En la Figura 7 se define mediante UML que cada Cuenta tiene exactamente un propietario y uno o más operadores. El propietario es la Persona que posee la cuenta y los operadores son otras Personas que tienen el derecho de retirar dinero y acceder a los detalles de la cuenta. Existe una restricción de negocio de que el propietario también debe ser un operador. Puede capturar esta restricción de la siguiente manera:

context Cuenta

inv propietarioEsOperador: self.operador->includes(self.propietario)

También podríamos haber escrito en *Persona* que *la cuentaPropietario* es un subconjunto de *cuentaOperada*:

```
context Persona
```

```
inv cuentaPropietarioSubConjuntoDeCuentaOperada:
```

```
    self.cuentaOperada->includesAll(self.cuentaPropietario)
```

Pre y post condiciones (pre y post)

Precondiciones y postcondiciones se aplican a operaciones. Su instancia contextual es una instancia de la clase a la que pertenecen las operaciones.

- Las precondiciones indican lo que debe ser cierto antes de que una operación se ejecute.
- Las postcondiciones indican lo que debe ser cierto después de que una operación se ejecute.

Considerando el ejemplo de la Figura 7, particularmente la operación *retirar(...)* que *CuentaCorriente* y *CajaDeAhorro* heredan de *Cuenta*. Existen dos reglas de negocio:

1. La cantidad a depositar debería ser mayor que cero.
2. Después de ejecutar la operación, la cantidad debería haberse añadido al saldo.

Puede expresar estas reglas de forma precisa en precondiciones y postcondiciones en la operación *Cuenta::depositar(...)* de la siguiente forma:

```
context Cuenta::depositar(cantidad: : Real): Real
```

```
pre cantidadAlIngresarMayorQueCero: cantidad > 0
```

```
post ingresoRealizado: self.saldo = self.saldo@pre + cantidad
```

La precondición *cantidadAlIngresarMayorQueCero* debe ser verdadera antes de que la operación se pueda ejecutar. Esto se asegura de que:

- No se pueden realizar ingresos de cantidad cero.
- No se pueden realizar ingresos de cantidades negativas.

La postcondición *ingresoRealizado* debe ser verdadera después de que la operación se haya ejecutado. Indica que el saldo original (*saldo@pre*) se incrementa en cantidad para obtener el saldo final. Observe el uso de la palabra clave *@pre*. Esta palabra clave se puede utilizar solamente dentro de postcondiciones. El saldo tiene un valor antes de que la operación se ejecute y otro valor después que la operación se ejecute. La expresión *saldo@pre* hace referencia al valor saldo antes de que la operación se ejecute. A menudo encuentra que necesita refernciar al valor original de algo en una postcondición.

Para completar la información, aquí tiene las restricciones en la operación *Cuenta::retirar(...)*.

```
context Cuenta::retirar (cantidad: Real)
```

```
pre cantidadARetirarMayorQueCero : cantidad > 0
```

```
post retiroRealizado: self.saldo = self.saldo@pre - cantidad
```

Ahora consideremos una operación `agregarOperador(nombre: String, id: String, dirección: String): Persona` que nos permite instanciar una nueva `Persona` y que sea incorporada como operador de la cuenta. La precondición verifica que no exista persona con el `id`, y la poscondición garantiza que la persona fue instanciada con la información de los argumentos de la operación, relacionada con la cuenta, y se retorna tal instancia.

```
context Cuenta::agregarOperador(nombre: String, id: String, dirección: String): Persona
pre: Persona.allInstances().id->excludes(id)
post:  result.ocllsNew() and
      result.nombre = nombre and id = result.id and dirección = result.direccion and
      self.operador= self.operador@pre->including(result)
```

Antes de dejar las precondiciones y postcondiciones, tenemos que considerar herencia. Cuando una operación se redefine por una subclase, obtiene las precondiciones y postcondiciones de la operación que redefine. Solamente puede cambiar estas condiciones de la siguiente manera:

- La operación redefinida solamente puede debilitar la precondición.
- La operación redefinida solamente puede fortalecer la postcondición.

Estas restricciones se aseguran de que se conserva el principio de sustitución.

Operaciones de consulta (body)

Puede utilizar OCL para especificar el resultado de una operación de consulta. Todas las operaciones `obtenerXXX()` en nuestro sencillo modelo de la Figura 7 son operaciones de consulta:

```
Cuenta::obtenerSaldo(): Real
Cuenta::obtenerPropietario(): Persona
Cuenta::obtenerOperadores(): Set(Persona)
CuentaCorriente::obtenerSaldoDisponible(): Real
CuentaCorriente::obtenerDescubiertoDisponible(): Real
```

Las expresiones OCL para las operaciones de consulta `Cuenta` son triviales y de hecho no le importará escribirlas. Se muestran más abajo a ejemplo:

```
context Cuenta::obtenerSaldo(): Real
body: self.saldo
context Cuenta::obtenerPropietario(): Persona
body: self.propietario
Cuenta::obtenerOperadores(): Set(Persona)
body: self.operador
```

Las operaciones de consulta de `CuentaCorriente` son más interesantes:

```

context CuentaCorriente::obtenerSaldoDisponible(): Real
body: -- pude retirar una cantidad hasta su límite de descubierto
      self.saldo + self.limiteDescubierto
context CuentaCorriente::obtenerDescubiertoDisponible(): Real
body:
      if self.saldo>0 then self.limiteDescubierto
      else self.saldo + self.limiteDescubierto endif

```

El valor de retorno de la operación es el resultado de evaluar la expresión OCL del body.

Navegación a y desde clases de asociación

Puede navegar a una clase de asociación al utilizar el nombre de la clase de asociación. Por ejemplo, considere la Figura 8. Puede expresar la operación de consulta obtenerTrabajos() de la siguiente forma:

```

@startuml
class Persona {
    nombre: String
    fechaNacimiento: Date
    obtenerEdad(): Integer
    obtenerTrabajos(): Trabajo[*]
    obtenerSueldoTotal(): Real
}
class Empresa
class Trabajo {
    nombre: String
    descripcion: String
    sueldo: Real
    obtenerEmpleado(): Persona
    obtenerEmpresa(): Empresa
}
Empresa "empresa 1..*" -- "empleado *" Persona
(Empresa, Persona).. Trabajo
hide circle
@enduml

```

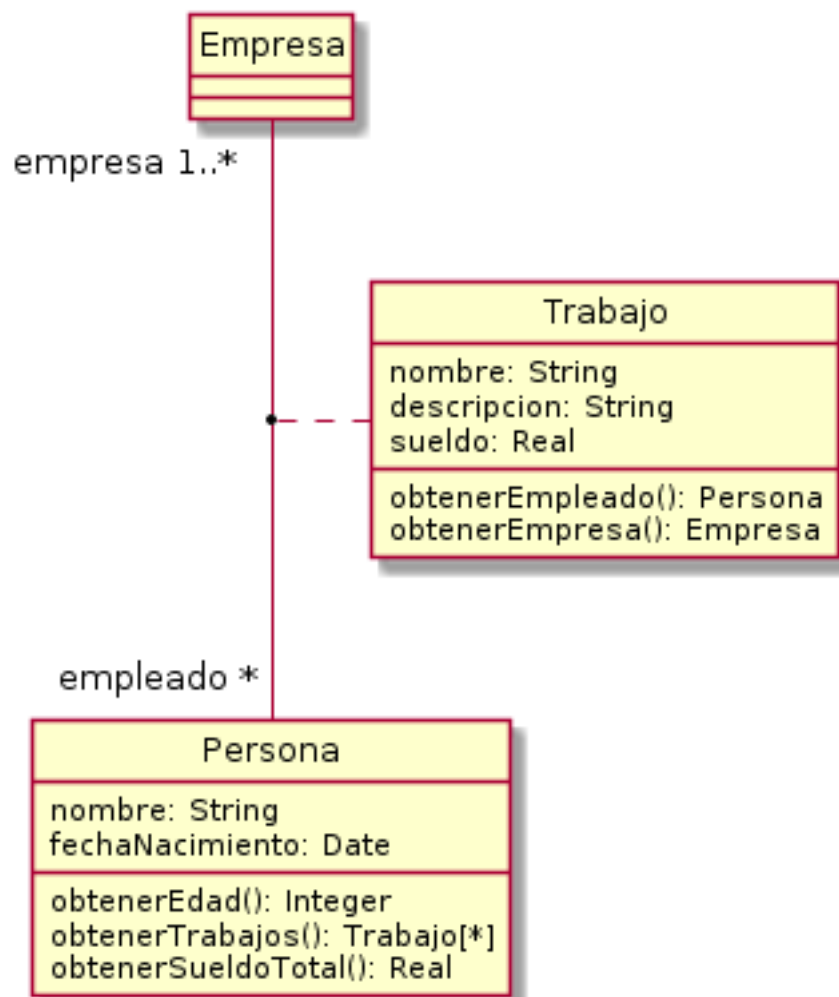


Figura 8.

La expresión `self.Trabajo` devuelve el Set de todos los objeto asociados con un objeto **Persona** dado. Tenga en cuenta que al navegar a una clase de asociación no usamos el nombre del extremo de asociación, sino que empleamos el nombre de la clase. Puede utilizar este Set de Trabajo (`self.Trabajo`) en expresiones OCL. Suponga que una regla de negocio indica que una **Persona** no puede tener dos Trabajos con el mismo nombre. Puede expresar esto en OCL de la forma:

```
context Persona
```

```
inv nombreTrabUnico: self.Trabajo->forAll(t1, t2| t1 <> t2 implies t1.nombre <> t2.nombre)
```

Esta regla también habría podido ser escrita como:

```
context Persona
```

```
inv nombreTrabUnico: self.Trabajo->isUnique(trabajo)
```

Suponga que la Empresa tiene un plan de prejubilación y hay una regla de negocio de que una Persona de más de 60 años no puede tener más de un Trabajo. Puede expresar esto en OCL de la siguiente manera:

```
context Persona
```

```
inv: self.obtenerEdad() > 60 implies self.Trabajo->size()=1
```

Para obtener el sueldo total de una Persona, necesita sumar el sueldo de cada Trabajo:

```
context Persona::obtenerSueldoTotal(): Real
```

```
body: self.Trabajo.sueldo->sum()
```

Puede navegar desde una clase de asociación al utilizar los nombres de los extremos de la asociación. Por ejemplo, aquí tiene la especificación en OCL para las operaciones de consulta obtenerEmpleado() y obtenerEmpresario() de Trabajo:

```
context Trabajo::obtenerEmpleado():Persona
```

```
body: self.empleado
```

```
context Trabajo::obtenerEmpresa(): Empresa
```

```
body: self.empresa
```

Asociaciones heredadas

Considere el modelo de la Figura 9, muestra un modelo para unidades de medida y sistemas de unidades (empleado en patrón de análisis cantidad).

```
@startuml
```

```
abstract class SistemaDeUnidades
```

```
class SistemaUniversal
```

```
class SistemaMetrico
```

```
abstract class Unidad
```

```
abstract class UnidadMetrica
```

```
abstract class UnidadUniversal
```

```
class Metro
```

```
class Centimetro
```

```
class Pie
```

```
class Pulgada
```

```
SistemaDeUnidades <|-- SistemaUniversal
```

```
SistemaDeUnidades <|-- SistemaMetrico
```

```
Unidad <|-- UnidadMetrica
```

```
Unidad <|-- UnidadUniversal
```



```

UnidadMetrica <|-- Metro
UnidadMetrica <|-- Centimetro
UnidadUniversal <|-- Pie
UnidadUniversal <|-- Pulgada
SistemaDeUnidades "sistema 1" -- "unidad 1..*" Unidad
hide members
hide circle
@enduml

```

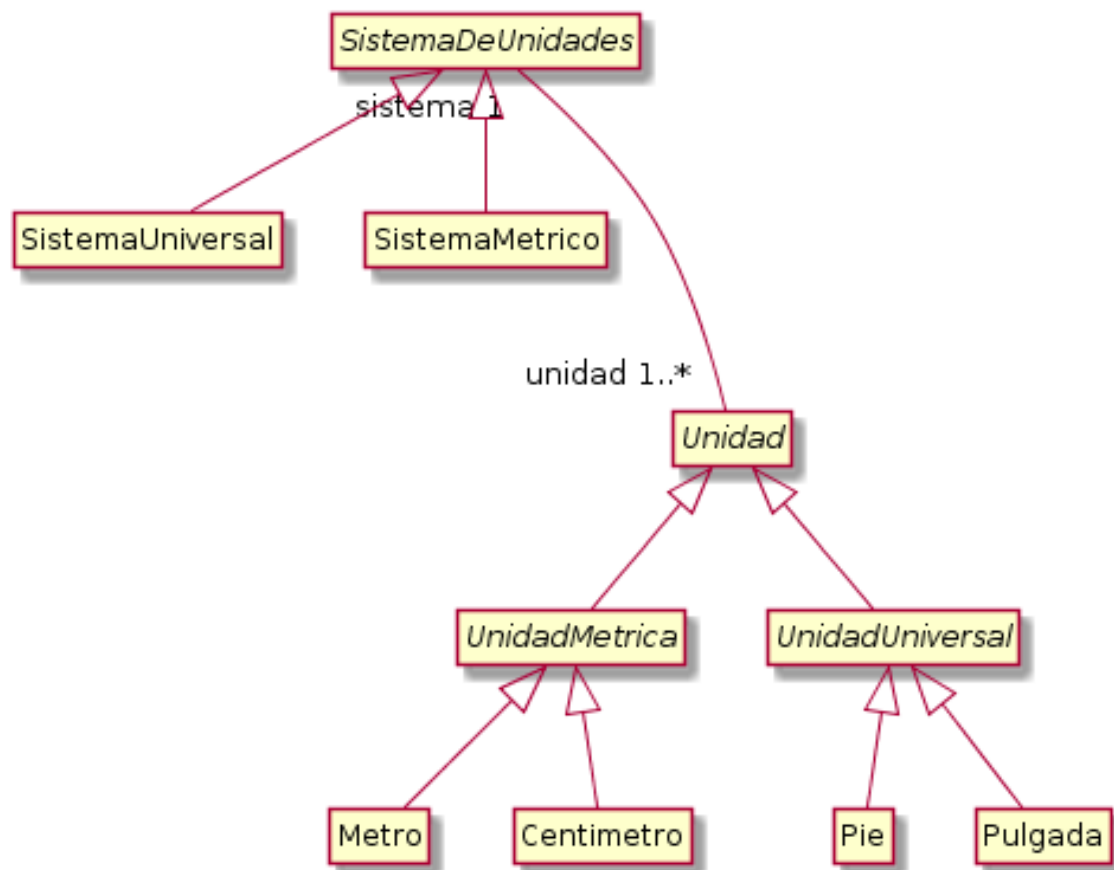


Figura 9.

Existen dos tipos diferentes de Unidad, UnidadMetrica y UnidadUniversal. Las unidades métricas pertenecen al sistema métrico (SistemaMetrico) y las unidades universales pertenecen al sistema universal (SistemaUniversal). Sin embargo, el modelo UML de la Figura 9 no representa esta situación, según el modelo cualquier unidad puede pertenecer a cualquier sistema. Puede hacer mas preciso el modelo definiendo restricciones OCL de la siguiente manera:

```

context UnidadMetrica
inv: self.sistema.ocllsTypeOf(SistemaMetrico)
context UnidadUniversal

```

```
inv: self.sistema.ocllsTypeOf(SistemaUniversal)
context SistemaMetrico
inv: self.unidad->forAll(u|u.ocllsKndOf(UnidadMetrica))
context SistemaUniversal
inv: self.unidad->forAll(u|u.ocllsKndOf(UnidadUniversal))
```

Bibliografía

Arlow, J., I. Neustadt. UML 2.0. Anaya, 2006.